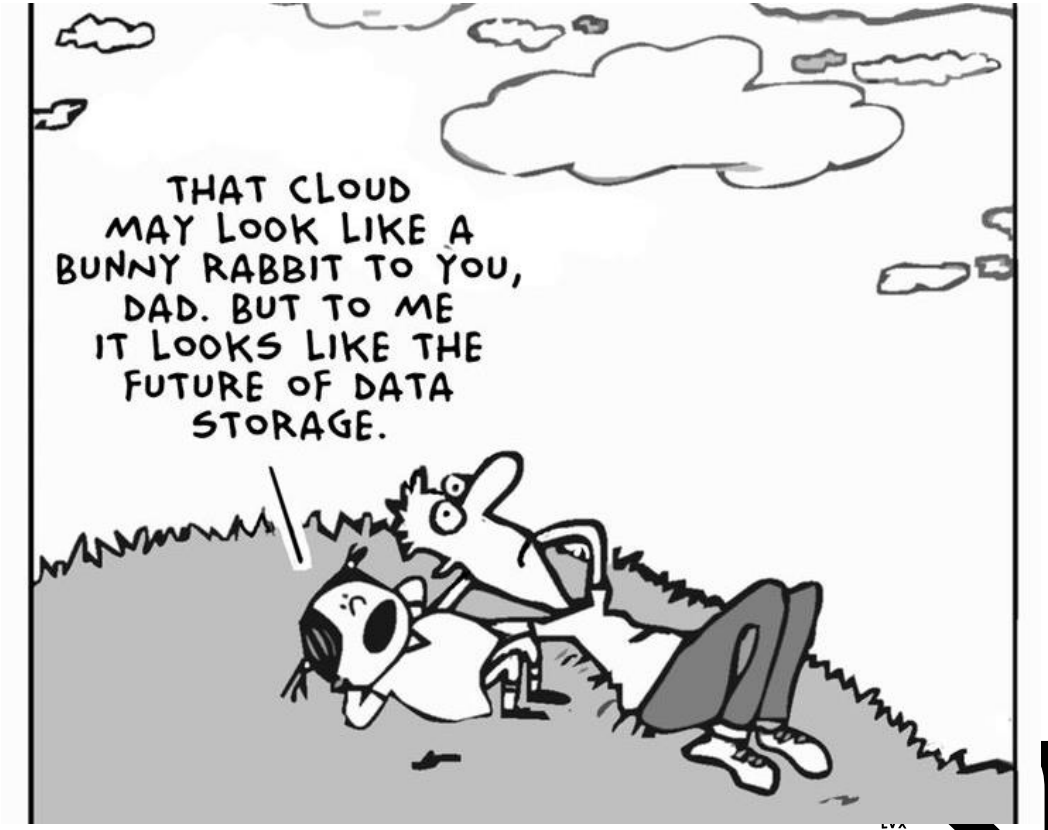
CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

Learning Objectives

- Distinguish between block-level storage, object storage, file storage, relational databases, and NoSQL databases and articulate when each is appropriate.
- Compare storage services across AWS, GCP, and Azure, understanding the naming differences and functional equivalents.
- Explain core database concepts including ACID properties, the CAP theorem, and consistency models.
- Make informed decisions about which cloud storage service to use for a given application requirement.
- Understand distribution models for databases including sharding, replication, and peer-to-peer architectures.



Cloud Storage Categories

Cloud Storage Categories

Cloud Storage at a Glance

Category	What It Is	Example Use
Block Storage	Virtual hard disks attached to VMs	OS disks, self-managed databases
File Storage	Shared file systems (NFS/SMB)	Shared content, web serving
Object Storage	Key-value blobs accessed via HTTP	Images, backups, static assets
Relational DB	Structured tables with SQL and ACID	User accounts, transactions
NoSQL DB	Flexible-schema stores for scale	Session data, catalogs, IoT

Modern cloud-native apps commonly use **multiple types simultaneously** — this is called **polyglot persistence**.

Cross-Cloud Naming

Same Concepts, Different Names

Category	AWS	GCP	Azure
Block Storage	EBS	Persistent Disk	Managed Disks
File Storage	EFS	Filestore	Azure Files
Object Storage	S3	Cloud Storage	Blob Storage
Managed RDBMS	RDS / Aurora	Cloud SQL / AlloyDB	Azure SQL Database
NoSQL (Key-Value / Document)	DynamoDB	Firestore / Bigtable	Cosmos DB

Block-Level Storage

The virtual hard drive of the cloud

What Is Block Storage?

- A **block** is a fixed-size sequence of bytes — the smallest addressable unit on a storage device.
- **How file access works:**
  - Application calls read("/path/to/file.txt")
  - OS forwards the request to the file system (ext4, NTFS, XFS, etc.)
  - File system translates the path → block address on disk
  - Data is returned to the application
- The OS is always the intermediary. Applications that use system calls to access files — like MySQL, PostgreSQL — **require** block storage.

Two Types of Block Storage

Network-Attached vs. Local (Instance) Storage

	Network-Attached	Instance (Local) Storage
Connection	Over the network	Physically on the host
Persistence	Independent of VM lifecycle	Lost when instance stops/terminates
Use case	Databases, OS disks	Caches, buffers, scratch data
AWS	EBS	Instance Store
GCP	Persistent Disk	Local SSD
Azure	Managed Disks	Temporary Disk / NVMe

Network-Attached Block Storage

- Key properties shared across all providers:
  - **Independent of VM** — volumes survive instance termination (if configured)
  - **Replicated** — automatically replicated within availability zone for durability
  - **Snapshots** — point-in-time backups, stored incrementally
  - **Encryption** — at rest and in transit, using provider key management
  - **Resizable** — can increase size without downtime (provider-specific caveats)
- Think of it as an **external hard drive** you plug into your virtual machine over a very fast network cable.

EBS Deep Dive

- **Amazon Elastic Block Store (EBS)**
  - Volumes attach to EC2 instances via network
  - Each volume scoped to a **single Availability Zone**
  - Designed for **99.999% availability**
  - One volume → one instance at a time (except io1/io2 Multi-Attach)
  - Volume types: gp3 (general purpose SSD), io2 (provisioned IOPS), st1 (throughput HDD), sc1 (cold HDD)
- **EBS volumes are NOT part of your EC2 instance.** You can terminate the instance and keep the volume.

EBS Snapshots

- **EBS / Persistent Disk / Managed Disk Snapshots**
  - Snapshots capture volume state at a point in time.
- **Incremental storage:**
  - First snapshot: copies all data (e.g., 100 GB)
  - Next snapshot: copies only changed blocks (e.g., 5 GB changed → you pay for 5 GB)
- Snapshots enable:
  - **Backup and recovery**
  - **Cross-region disaster recovery** (copy snapshots to other regions)
  - **Creating new volumes** from a snapshot (effectively cloning a disk)
  - **Building machine images** (AMIs on AWS use EBS snapshots)

Instance Store

- **Ephemeral Local Storage**
  - Disks **physically attached** to the host machine
  - Named ephemeral0 through ephemeral23 on AWS
  - **No network overhead** → lower latency, no noisy neighbor problem
- **Data persists through:** reboots
- **Data is LOST when:**
  - Instance stops
  - Instance terminates
  - Underlying disk fails
- **Rule:** Never use instance store for data you cannot afford to lose.
- **Cost:** Included in the VM price (AWS, Azure). Billed separately on GCP (Local SSD).

Block Storage Comparison

Feature	AWS EBS	GCP Persistent Disk	Azure Managed Disks
Replication	Within AZ	Within zone or across 2 zones (Regional PD)	Within datacenter (LRS) or across zones (ZRS)
Snapshots	Stored in S3	Stored in Cloud Storage	Stored in Azure Storage
Encryption	KMS	Default + Cloud KMS	Platform or customer keys
Max size	64 TiB	64 TiB	64 TiB
Multi-attach	io1/io2 only	Multi-writer mode	Ultra / Premium SSD v2

File Storage

File Storage Overview

EFS / Filestore / Azure Files

File storage = a **shared file system** that multiple VMs can mount simultaneously.

Think of it as a **network drive at the office** that everyone on the team can access.

Feature	AWS EFS	GCP Filestore	Azure Files
Protocol	NFS v4.0/4.1	NFS	**SMB and NFS**
Auto-scaling	Yes (automatic)	No (provisioned)	Yes (up to limit)
Multi-AZ	Yes (default)	Enterprise tier only	ZRS/GRS options
Windows support	No (use FSx)	No	**Yes (SMB)**
Max size	Petabytes	100 TiB	100 TiB per share

EFS Key Properties

- **Amazon Elastic File System**
  - Supports **NFS v4.0 and v4.1**
  - Stores data across **multiple Availability Zones** automatically
  - **Scales automatically** — grows from empty to petabytes with no provisioning
  - Multiple EC2 instances mount the same file system → **shared data source**
  - Designed for multi-threaded applications with high aggregate throughput
- **Limitations:**
  - Cannot be used as the **root volume** for EC2
  - Linux only (for Windows, use Amazon FSx)
- **Use cases:** Big data analytics, media processing, content management, web serving, home directories

Object Storage

Objects vs. Files

- In an object store, every piece of data is an **object** with three components.
  1. Key
  2. Metadata
  3. Data
- Key differences from a file system:
  - **Flat namespace** — no real directories (slashes in keys are just characters)
  - **Accessed via HTTP APIs** — GET, PUT, DELETE (not OS system calls)
  - Metadata is separated from data → clients can query metadata without loading the data

Component	Description	Example
Key	Globally unique identifier	`/images/2025/profile.png`
Metadata	Descriptive info about the object	Content-type, tags, permissions, size, date
Data	The actual bytes	The image file itself

Object Storage Services

S3 / Cloud Storage / Blob Storage

Feature	AWS S3	GCP Cloud Storage	Azure Blob Storage
Max object size	5 TB	5 TB	~4.75 TiB
Consistency	**Strong**	**Strong**	**Strong**
Encryption	Default (SSE-S3)	Default (Google-managed)	Default (Microsoft-managed)
Versioning	Yes	Yes	Yes
Event notifications	SNS, SQS, Lambda	Pub/Sub, Eventarc	Event Grid
Static website hosting	Yes	Yes	Yes

Buckets and Containers

- **Organizing Objects**
  - **AWS & GCP:** Objects live in **Buckets**
  - **Azure:** Objects (blobs) live in **Containers** within a **Storage Account**
- Key facts about buckets:
  - **Globally unique names** — no two customers anywhere can share a bucket name
  - Buckets have properties: region, access permissions, versioning, logging
  - Buckets can send **event notifications** when objects are created, modified, or deleted
- **Event-driven example:** User uploads image to S3 → bucket triggers Lambda function → Lambda generates thumbnail → thumbnail saved back to S3

Storage Classes

Hot, Warm, Cold, and Archive Tiers

Tier	AWS S3	GCP Cloud Storage	Azure Blob
Hot (frequent access)	Standard	Standard	Hot
Warm (infrequent)	Standard-IA	Nearline (30-day min)	Cool (30-day min)
Cold (rare access)	Glacier Instant Retrieval	Coldline (90-day min)	Cold (90-day min)
Archive (long-term)	Glacier Deep Archive	Archive (365-day min)	Archive
Auto-tiering	Intelligent-Tiering	Autoclass	Access tier management

Pattern: Storage cost decreases as you go colder, but retrieval cost increases.

All tiers share **11 nines (99.999999999%) durability**.

Lifecycle Policies

- **Automate Data Movement -** Lifecycle rules automatically transition or delete objects based on age or conditions.
- **Example policy for a photo-sharing app:**
  - Day 0–30: Standard tier (frequently viewed)
  - Day 31–90: Infrequent Access tier (lower storage cost)
  - Day 91–365: Archive tier (rarely accessed, cheap storage)
  - Day 366+: Delete permanently
- All three providers support lifecycle policies. Combined with versioning, this controls both current and previous versions of objects.

Versioning

- **Protecting Against Accidental Overwrites**
- **Without versioning (default):**
  - Upload object with key A and data 1
  - Upload object with key A and data 2
  - Download key A → you get 2. Data 1 is gone forever.
- **With versioning enabled:**
  - Upload key A data 1 → stored as version 1
  - Upload key A data 2 → stored as version 2
  - Download key A → you get 2 (latest)
  - Download key A version 1 → you get 1 (still available)
- **Warning:** Every version consumes storage and costs money.

Object Store Encryption Options

Server-Side Encryption

Method	Who Manages Keys	AWS Name
Provider-managed keys	AWS / Google / Microsoft	SSE-S3 (default)
Cloud KMS keys	You manage keys in cloud KMS	SSE-KMS
Customer-provided keys	You supply the key with each request	SSE-C

All three providers **encrypt at rest by default** using provider-managed keys.

All support upgrading to customer-managed keys for additional control (audit trails, key rotation, key disablement).

Cross-Region Replication

Geographic Redundancy

Provider	Feature	How It Works
AWS	Cross-Region Replication (CRR)	Async copy from source bucket → destination bucket in another region
GCP	Dual-Region / Multi-Region buckets	Automatic replication at bucket creation
Azure	GRS / GZRS	Automatic async replication to a paired region

Use cases: Disaster recovery, compliance (data residency), lower latency for global users

Object Storage Is NOT…

- **Not a file system:**
  - Flat namespace, not POSIX-compliant
  - Tools like s3fs and gcsfuse exist but have significant performance and consistency limitations
  - Use EFS / Filestore / Azure Files instead
- **Not a database:**
  - No query engine — you must know the exact bucket + key
  - No transactions, no indexing, no joins
  - Pair with DynamoDB, RDS, or Cloud SQL to index and query object metadata
- **Not for system-call-based applications:**
  - MySQL, PostgreSQL, etc. cannot store data directly in S3

Relational Databases

What Is a Relational Database?

- **Tables, Rows, Columns, SQL**
  - Concept introduced by **E. F. Codd at IBM in 1970**
  - Data organized as **tables** (relations) with rows and columns
  - Queried using **SQL** (Structured Query Language)
  - Connected via **foreign keys** and combined with **joins**
- **Popular engines:** MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, SQLite
- **Cloud-native engines:** Amazon Aurora, GCP AlloyDB, Azure SQL Database

ACID Transactions

The Four Guarantees

Property	Guarantee	Bank Transfer Example
Atomicity	All or nothing	Debit + credit both succeed, or neither does
Consistency	Valid state → valid state	Account balance never goes negative (if constrained)
Isolation	Concurrent txns don't interfere	Two transfers to same account produce correct total
Durability	Committed = permanent	Committed transfer survives a server crash

ACID is what makes relational databases trustworthy for financial transactions, inventory management, and any data where correctness matters.

Managed Database Services

RDS / Cloud SQL / Azure SQL

Feature	AWS RDS / Aurora	GCP Cloud SQL / AlloyDB	Azure SQL Database
Engines	MySQL, PostgreSQL, MariaDB, Oracle, SQL Server	MySQL, PostgreSQL, SQL Server	SQL Server (Azure SQL), MySQL, PostgreSQL
Cloud-native engine	Aurora (MySQL/PG compatible)	AlloyDB (PG compatible)	Azure SQL (SQL Server based)
Automated backups	Yes (daily snapshot + transaction logs)	Yes (configurable retention)	Yes (up to 35 days + long-term)
Point-in-time recovery	Yes (up to 35-day retention)	Yes (up to 365 days)	Yes
Encryption at rest	KMS	Default + Cloud KMS	TDE (default)

"Managed" means: Provider handles patching, backups, failover, replication. You handle schema design, queries, security groups, and instance sizing.

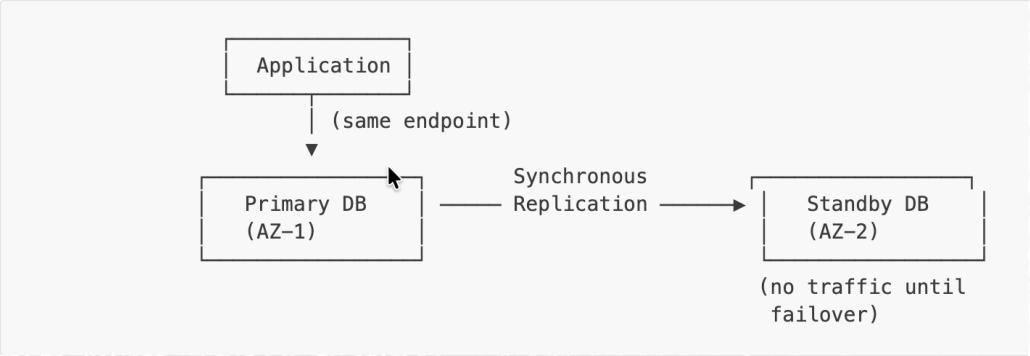
Backups and Recovery

- **Two Types of Backups (AWS RDS)**
  - **Automated Backups:**
    - Full daily snapshot + continuous transaction logs
    - Point-in-time recovery to any second within retention period
    - Enabled by default, stored in S3 at no extra charge
    - **Deleted when you terminate the RDS instance**
  - **Manual Snapshots:**
    - User-initiated at any time
    - **Persist even after instance deletion**
- **Important:** Restoring any backup creates a **new database instance** with a **new endpoint**. You cannot restore in-place over an existing instance.

Multi-AZ Deployments

- Primary fails → **automatic failover** to standby
- **Same DNS endpoint** — application reconnects without code changes
- Available on AWS (RDS Multi-AZ), GCP (Cloud SQL HA), Azure (zone-redundant)

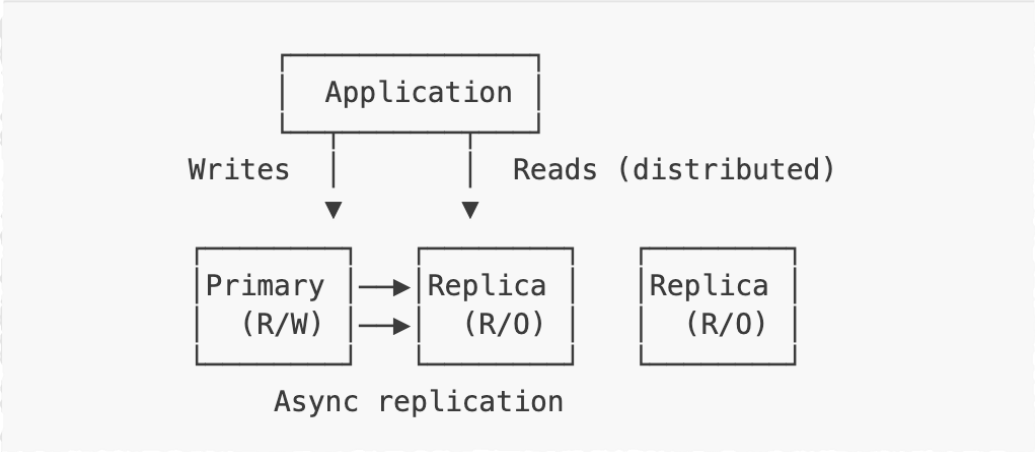
High Availability via Synchronous Replication



Read Replicas

- **Scaling Read-Heavy Workloads**
  - **Asynchronous** replication — replicas may be slightly behind primary
  - Can be **promoted** to standalone primary instance
  - AWS Aurora: up to 15 replicas. RDS: up to 5.
- **Multi-AZ vs. Read Replicas:**
  - Multi-AZ = **availability** (standby doesn't serve traffic until failover)
  - Read Replicas = **performance** (actively serve read queries)

Scaling Read-Heavy Workloads



NoSQL Databases

3 DATABASE ADMINS
WALKED INTO
A NOSQL BAR...

A LITTLE WHILE LATER
THEY WALKED OUT BECAUSE
THEY COULDN'T FIND A TABLE

Limitations of Relational Databases at Scale

- **Limitations of Relational Databases at Scale**
  - **Problem 1: Impedance Mismatch**
    - App data lives in memory as objects, lists, nested structures
    - Relational tables are flat rows with foreign keys
    - Significant mapping code required → common source of bugs
  - **Problem 2: Horizontal Scaling**
    - Relational DBs scale **up** (bigger server) more easily than **out** (more servers)
    - Write throughput becomes a bottleneck at massive scale
- **NoSQL databases** address both: flexible data models that match application structures, and architectures designed to scale horizontally across clusters.

NoSQL Characteristics - What Makes a Database "NoSQL"?

- **Schema-free:** No predefined structure — different records can have different fields
- **Non-relational:** Does not use tables/rows/foreign keys model
- **Cluster-friendly:** Designed to run across many servers
- **No joins:** Related data is denormalized (stored together) rather than joined at query time
- **Eventual consistency (often):** Trades strict consistency for availability and performance
- The term "NoSQL" originated as a **hashtag** for a 2009 meetup — chosen because it was short, memorable, and had few Google results.

Types of NoSQL Databases

Four Major Data Models

Type	Model	Strengths	Examples
Key-Value	Key → arbitrary blob	Extreme throughput, low latency	DynamoDB, Redis, Memcached
Document	Key → structured JSON/BSON	Queryable fields within documents	MongoDB, Firestore, DocumentDB
Column-Family	Row key → column families	Analytics, time-series, wide rows	Bigtable, Cassandra, Keyspaces
Graph	Nodes + edges	Traversing relationships	Neptune, Neo4j, Cosmos DB (Gremlin)

Choose the type based on your **access pattern**, not tradition.

Key-Value Stores

- Present a key → get back a blob of data
- Usually **no query language** — lookup by exact key only
- Extreme performance: sub-millisecond reads at scale
- **Cloud services:** AWS DynamoDB, GCP Memorystore (Redis), Azure Cache for Redis, Azure Cosmos DB (Table API)

```
Key (string)              Value (any type)
____________              ________________

"user:1234"       →       { name: "Alice", email: "..." }
"session:abcd"    →       { token: "xyz", expires: "..." }
"config:app"      →       { theme: "dark", lang: "en" }
```

Document Databases

- Unlike key-value: you can **query on any field** (e.g., find all orders where status = "shipped")
- **Cloud services:** AWS DocumentDB, GCP Firestore, Azure Cosmos DB, MongoDB Atlas

```
{
  "_id": "order-5678",
  "customer": "Alice",
  "items": [
    { "product": "Laptop", "qty": 1, "price": 999.99 },
    { "product": "Mouse",  "qty": 2, "price": 29.99 }
  ],
  "total": 1059.97,
  "status": "shipped"
}
```

Column-Family and Graph Databases

- **Column-Family (Wide-Column)**
  - Data organized by **column families** rather than rows
  - Efficient for reading a few columns across many rows (analytics, time-series)
  - Think: a spreadsheet where each row can have a different set of columns
  - **Examples:** Google Bigtable, Apache Cassandra, AWS Keyspaces
- **Graph**
  - Data modeled as **nodes** (entities) and **edges** (relationships)
  - Optimized for traversing connections (friends-of-friends, shortest path)
  - **Examples:** AWS Neptune, Azure Cosmos DB (Gremlin API), Neo4j

NoSQL Across Cloud Providers

Managed NoSQL Services

Type	AWS	GCP	Azure
Key-Value / Document	DynamoDB	Firestore	Cosmos DB
Wide-Column	Keyspaces (Cassandra)	Cloud Bigtable	Cosmos DB (Cassandra API)
In-Memory Cache	ElastiCache (Redis)	Memorystore (Redis)	Azure Cache for Redis
Graph	Neptune	Neo4j (Marketplace)	Cosmos DB (Gremlin API)

Azure Cosmos DB is notable for supporting **multiple APIs** (SQL, MongoDB, Cassandra, Gremlin, Table) through a single service.

CAP Theorem

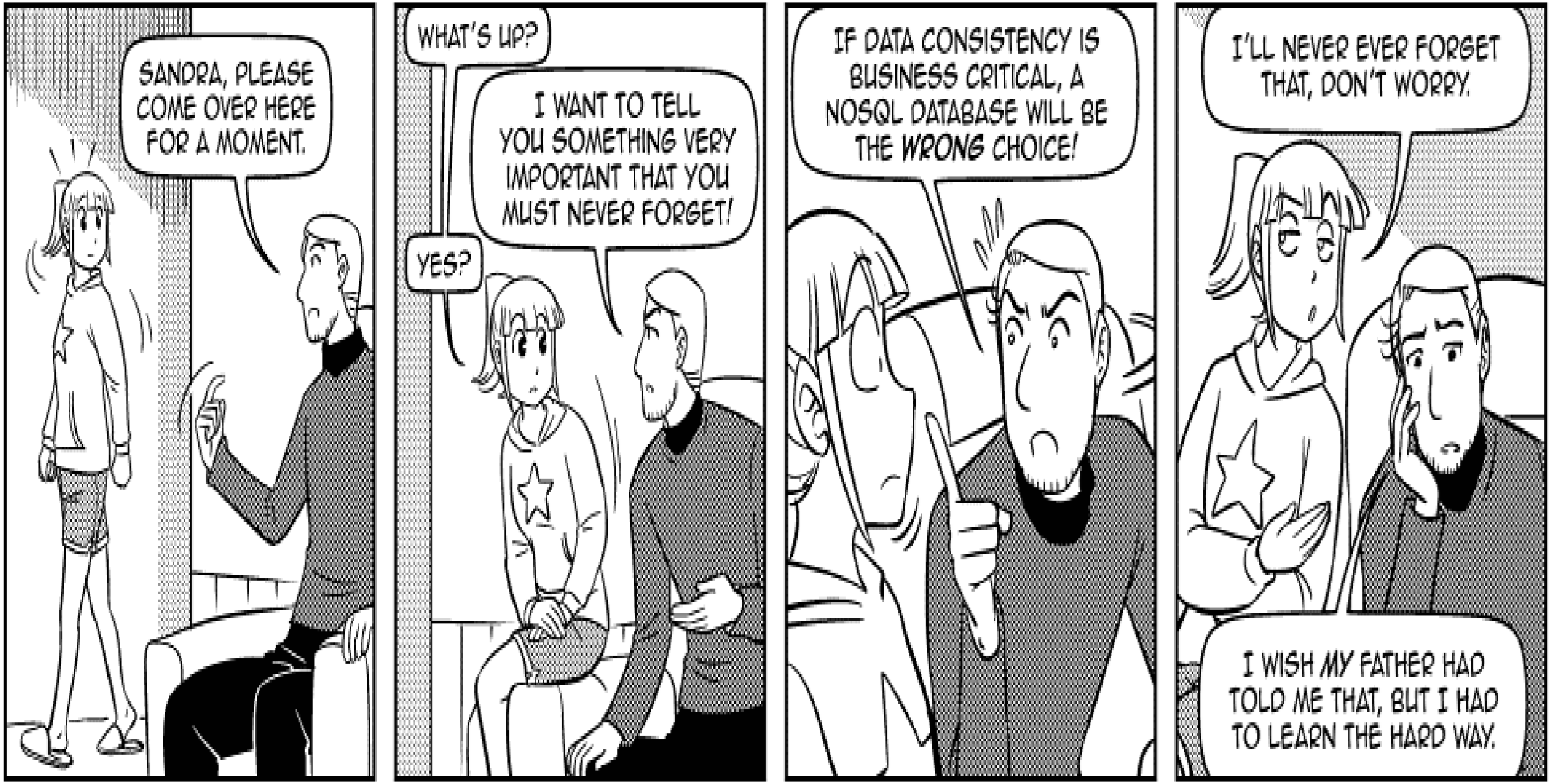
- **Consistency:** Every read returns the most recent write
- **Availability:** Every request gets a response (even if not the latest data)
- **Partition Tolerance:** System works despite network failures between nodes
- In practice, network partitions **do happen**, so you choose between **C** and **A** during a partition.
- **RDBMS → favors Consistency** (refuses requests rather than returning stale data)
- **Many NoSQL → favors Availability** (returns a response, possibly stale)
- ⚠️ CAP "Consistency" ≠ ACID "Consistency" — they describe different things.

Pick Two (When a Partition Happens)

```
                Consistency
                   /\
                  /  \
                 /    \
                / Pick  \
               /  Two    \
              /___________\
Availability    Partition
                Tolerance
```

Eventual Consistency - The NoSQL Trade-Off

- **Eventual consistency:** Changes propagate to all nodes "eventually" (typically milliseconds). During that window, reads might return stale data.
- **Acceptable for:** Social media like counts, product view counts, activity feeds
- **Not acceptable for:** Bank balances, inventory for limited-stock items, medical records
- **Strong consistency** is available in NoSQL too (e.g., DynamoDB strongly consistent reads, Cosmos DB strong consistency level) — but it comes with higher latency and lower throughput.



Sandra and Woo by Oliver Knörzer (writer) and Powree (artist) – www.sandraandwoo.com

Database Considerations

Concurrency Control - Optimistic vs. Pessimistic

Approach	Strategy	Best When
Optimistic	Allow operations freely, check for conflicts at commit time. Abort and retry if conflict detected.	Conflicts are rare
Pessimistic	Lock the data before modifying. Other transactions wait.	Conflicts are frequent

Version Stamps: A field that changes with every update. Read the version, then check it again before writing. If it changed → someone else modified the data → resolve the conflict.

Most NoSQL systems favor **optimistic** concurrency.

Write-Write and Read-Write Conflicts

- **Write-Write Conflict:** Two users update the same record simultaneously with different values.
- Optimistic approaches:
  - **Conditional update** — check if value changed since last read before writing
  - **Accept both** — record that a conflict exists, resolve later
- **Read-Write Conflict:** Alice reads 1 ticket left, hesitates. Bob reads 1 ticket left, buys it. Alice tries to buy → ticket is gone.
- Solved by version stamps, conditional writes, or pessimistic locking depending on the system.

Distribution Models - How NoSQL Spreads Data Across Servers

- **Single Server:** All reads/writes on one machine. Simple but limited. Good for development.
- **Sharding:** Split data across nodes by key. Different users → different servers. Improves read AND write performance. ⚠ Wrong shard key → hot spots.
- **Primary-Secondary Replication:** One primary handles writes, secondaries get copies for reads. Primary = bottleneck and potential single point of failure.
- **Peer-to-Peer Replication:** All nodes accept writes. No single point of failure. Requires conflict resolution.
- **Combined:** Sharding + Replication together. Common in production column-family databases (e.g., Cassandra).

Normalization vs. Denormalization

- **Relational Thinking vs. NoSQL Thinking**
- **Normalization (Relational):**
  - Eliminate redundancy → data in one place
  - Requires **joins** to reassemble related data
  - Update once → consistent everywhere
  - Optimized for **write correctness**
- **Denormalization (NoSQL):**
  - Store related data **together** (embed/duplicate)
  - No joins needed → fast reads
  - Update name? Must update in **every** document that contains it
  - Optimized for **read performance**
- NoSQL databases don't support joins → **denormalization is the standard approach**.

Polyglot Persistence

Use the Right Database for Each Job

Example: E-Commerce Application

Data	Storage Choice	Why
User accounts, orders	PostgreSQL (RDS / Cloud SQL)	ACID transactions, complex queries
Product images	S3 / Cloud Storage / Blob Storage	Unlimited, cheap, HTTP access
Shopping cart	DynamoDB / Firestore / Cosmos DB	Fast key-value access, flexible schema
Session tokens	Redis (ElastiCache / Memorystore)	Sub-millisecond in-memory access
Product search	Elasticsearch / OpenSearch	Full-text search and filtering

Cloud managed services make this practical — you don't operate each engine yourself.

Choosing the Right Storage

- Need OS file system access?
- └─ Yes → Block Storage (EBS / Persistent Disk / Managed Disks)
- └─ No ↓
- Multiple VMs need shared file access?
- └─ Yes → File Storage (EFS / Filestore / Azure Files)
- └─ No ↓
- Unstructured data (images, video, backups)?
- └─ Yes → Object Storage (S3 / Cloud Storage / Blob Storage)
- └─ No ↓
- Need ACID transactions and complex SQL queries?
- └─ Yes → Relational Database (RDS / Cloud SQL / Azure SQL)
- └─ No ↓
- Need flexible schema and horizontal scale?
- └─ Yes → NoSQL Database (DynamoDB / Firestore / Cosmos DB)

Summary

Key Takeaways

- **Block storage** = virtual hard disks for VMs and databases (EBS, Persistent Disk, Managed Disks)
- **File storage** = shared file systems for multiple instances (EFS, Filestore, Azure Files)
- **Object storage** = unlimited HTTP-accessible storage for unstructured data (S3, Cloud Storage, Blob Storage)
- **Relational DBs** = ACID-compliant structured storage with SQL (RDS, Cloud SQL, Azure SQL)
- **NoSQL DBs** = flexible schemas with horizontal scalability (DynamoDB, Firestore, Cosmos DB)
- **CAP theorem** = you choose between consistency and availability during network partitions
- **Polyglot persistence** = use multiple storage types in one application — the cloud-native way

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS